

Lab 4: Browser security

Introducción

Este laboratorio te presentará ataques basados en el navegador, así como la manera de prevenirlos. El laboratorio tiene varias partes:

- Parte 1: ataque de cross-site scripting
- Parte 2: ataque de canal lateral y phishing
- Parte 3: un gusano de perfil

Cada parte incluye varios ejercicios que te ayudan a construir un ataque. Todos los ataques implicarán explotar debilidades en el sitio de zoobar, pero son representativos de las debilidades que se encuentran en sitios web reales.

Configuración de la red

Para este laboratorio, elaborarás ataques en tu navegador web que explotan vulnerabilidades en la aplicación web de zoobar. Para asegurarnos de que tus exploits funcionen en nuestras máquinas cuando califiquemos tu laboratorio, debemos acordar la URL que hace referencia al sitio web de zoobar. Para los fines de este laboratorio, tu sitio web de zoobar debe estar ejecutándose en `http://localhost:8080/`. Si has estado usando la dirección IP de tu VM, como `http://192.168.177.128:8080/`, no funcionará en este laboratorio.

Si estás usando KVM o VirtualBox, las instrucciones que proporcionamos en el laboratorio 1 ya garantizan que el puerto 8080 en localhost se reenvíe al puerto 8080 en la máquina virtual. Si estás usando VMware, utilizaremos la función de reenvío de puertos de ssh para exponer el puerto 8080 de tu VM como `http://localhost:8080/`. Primero encuentra la dirección IP de tu VM.

Para hacerlo, inicia sesión como root en la consola, ejecuta `ip addr show dev eth0`, y anota la dirección IP que aparece junto a inet. (Esta es la misma dirección IP que has estado usando en los laboratorios anteriores). Luego configura el reenvío de puertos SSH de la siguiente manera (lo cual depende de tu cliente SSH):

Para usuarios de Mac y Linux: abre una terminal *en tu máquina* (no en tu VM) y ejecuta

```
$ ssh -L localhost:8080:localhost:8080 student@VM-IP-ADDRESS
student@VM-IP-ADDRESS's password: 6858
```

Para usuarios de Windows, esta debería ser una opción en tu cliente SSH. En PuTTY, sigue estas instrucciones. Usa 8080 como puerto de origen y localhost:8080 como puerto remoto.

El reenvío permanecerá activo mientras la conexión SSH esté abierta.

Navegador web

En este laboratorio, te recomendamos usar la versión actual del navegador Mozilla Firefox para desarrollar tus ataques. Existen diferencias sutiles (y no tan sutiles) en la forma en que distintos navegadores manejan HTML, JavaScript y cookies, y algunos ataques que funcionan en Internet Explorer o Chrome (por ejemplo) pueden no funcionar en Firefox, y viceversa. Sugerimos Firefox porque está ampliamente disponible y puede ejecutarse en una variedad de sistemas operativos.

Si tienes una extensión de bloqueo de anuncios en tu navegador web, como uBlock, puede que quieras desactivarla para evitar que interfiera con los ataques que desarrollarás en este laboratorio.

Configuración del servidor web

Antes de comenzar a trabajar en estos ejercicios, por favor usa Git para hacer commit de tus soluciones del Laboratorio 3, obtén la versión más reciente del repositorio del curso y luego crea una rama local llamada lab4 basada en nuestra rama lab4, origin/lab4. No fusionar tus soluciones de los laboratorios 2 y 3 en el laboratorio 4. Estos son los comandos de shell:

```
student@vm-6858:~$ cd lab
student@vm-6858:~/lab$ git commit -am 'my solution to lab3'
[lab3 c54dd4d] my solution to lab3
1 files changed, 1 insertions(+), 0 deletions(-)
student@vm-6858:~/lab$ git pull
Already up-to-date.
student@vm-6858:~/lab$ git checkout -b lab4 origin/lab4
Branch lab4 set up to track remote branch lab4 from origin.
Switched to a new branch 'lab4'
```

```
student@vm-6858:~/lab$ make
...
```

Ten en cuenta que el código fuente del laboratorio 4 se basa en el servidor web inicial del laboratorio 1. No incluye separación de privilegios ni perfiles de Python.

Ahora puedes iniciar el servidor web **zookws** de la siguiente manera.

```
student@vm-6858:~/lab$ ./zookd 8080
```

Abre tu navegador y ve a la URL <http://localhost:8080/>. Deberías ver la aplicación web de zoobar. Si no la ves, regresa y verifica nuevamente tus pasos. Si no logras que el servidor web funcione, ponte en contacto con el personal del curso antes de continuar.

Elaboración de ataques

Crearás una serie de ataques contra el sitio web **zoobar** con el que has trabajado en laboratorios anteriores. Estos ataques explotan vulnerabilidades en el diseño y la implementación de la aplicación web. Cada ataque presenta un escenario distinto con objetivos y restricciones únicos, aunque en algunos casos podrías reutilizar partes de tu código.

Ejecutaremos tus ataques después de limpiar la base de datos de usuarios registrados (excepto el usuario llamado “attacker”), así que no asumas la presencia de otros usuarios en tus ataques enviados.

Puedes ejecutar nuestras pruebas con **make check**; esto ejecutará tus ataques contra el servidor y te dirá si tus exploits están funcionando correctamente. Como en laboratorios anteriores, ten en cuenta que las verificaciones realizadas por **make check** no son exhaustivas, especialmente respecto a condiciones de carrera. Podrías querer ejecutar las pruebas varias veces para convencerte de que tus exploits son robustos.

Los ejercicios 5, 13 y 14 requieren que el sitio mostrado se vea de cierta manera. El script **make check** no es lo suficientemente inteligente como para comparar cómo se ve el sitio con y sin tu ataque, así que tendrás que hacer esa comparación tú mismo (y nosotros también, durante la calificación). Cuando **make check** se ejecuta, genera imágenes de referencia de cómo *se supone* que debe verse la página del ataque (**answer-XX.ref.png**) y de lo que tu página de ataque realmente muestra (**answer-XX.png**), y las coloca en el directorio **lab4-tests/**. Asegúrate de que tus capturas de pantalla **answer-XX.png** se parezcan a las imágenes de referencia en **answer-XX.ref.png**. Para ver estas imágenes desde **lab4-tests/**, cópialas a tu máquina local o ejecuta **python3 -m http.server 8080** y visualiza las imágenes visitando <http://localhost:8080/lab4-tests/>. *Ten en cuenta que http.server de Python cachea las respuestas*, así que deberías detenerlo y reiniciarlo después de ejecutar **make check**.

Parte 1: un ataque de cross-site scripting (XSS)

La página de usuarios de zoobar tiene una falla que permite el robo de la cookie de un usuario autenticado desde su navegador, si un atacante logra engañar al usuario para que haga clic en una URL especialmente construida por el atacante. Tu trabajo es construir dicha URL. Un atacante podría enviar la URL por correo electrónico a la víctima, esperando que la víctima haga clic en ella. Un atacante real podría usar una cookie robada para hacerse pasar por la víctima.

Desarrollarás el ataque en varios pasos. Para aprender la infraestructura necesaria para construir los ataques, primero realiza algunos ejercicios que te familiaricen con JavaScript, el DOM, etc.

Ejercicio 1: Imprimir la cookie.

Las cookies son el mecanismo principal de HTTP para rastrear usuarios a través de las solicitudes. Si un atacante logra obtener la cookie de otro usuario, puede hacerse pasar completamente por ese otro usuario. Para este ejercicio, tu objetivo es simplemente imprimir la cookie del usuario que ha iniciado sesión cuando accede a la página “Users”.

1. Lee acerca de cómo se accede a las cookies desde JavaScript.
2. Guarda una copia de **zoobar/templates/users.html** (necesitarás restaurar esta versión original más adelante). Agrega una etiqueta **<script>** a **users.html** que imprima la cookie del usuario autenticado usando **alert()**. Es posible que tu script no funcione inmediatamente si cometiste un error de programación en JavaScript. Afortunadamente, Chrome tiene herramientas de depuración fantásticas disponibles en el Inspector: la consola de JavaScript, el inspector del DOM y el monitor de red. La consola de JavaScript te permite ver qué excepciones se están generando y por qué. El inspector del DOM te permite observar la estructura de la página y las propiedades y métodos de cada nodo que contiene. El monitor de red te permite inspeccionar las solicitudes entre tu navegador y el sitio web. Al hacer clic en una de las solicitudes, puedes ver qué cookie está enviando tu navegador y compararla con lo que imprime tu script.

3. Coloca el contenido de tu script en un archivo llamado `answer-1.js`. Tu archivo solo debe contener JavaScript (no incluyas etiquetas `<script>`).

Ejercicio 2: Registrar la cookie.

Modifica tu script para que registre la cookie del usuario para el atacante usando el script de registro. El ataque debe seguir activándose cuando el usuario visite la página “Users”.

Por favor revisa las instrucciones en <https://css.csail.mit.edu/6.858/2022/labs/log.php> y usa esa URL en tus scripts para registrar la cookie robada. Puedes registrar tantas veces como quieras mientras trabajas en el proyecto, pero por favor no ataques ni abusos del script de registro. Ten en cuenta que la cookie tiene caracteres que probablemente necesiten ser codificados en la URL. Echa un vistazo a `encodeURIComponent` y `decodeURIComponent`.

Cuando tengas un script funcional, ponlo en un archivo llamado `answer-2.js`. Nuevamente, tu archivo solo debe contener JavaScript (no incluyas etiquetas `<script>`).

Ejercicio 3: Ejecución remota.

En este ejercicio, tu objetivo es construir una URL que, al accederse, haga que el navegador de la víctima ejecute JavaScript que tú, como atacante, hayas proporcionado. En particular, queremos que crees una URL que contenga un fragmento de código en uno de los parámetros de consulta que, debido a un bug en zoobar, la página “Users” envía de vuelta al navegador. El código se ejecutará entonces como JavaScript en el navegador. Esto se conoce como “cross-site scripting reflejado” y es una vulnerabilidad muy común en la Web hoy en día.

Para este ejercicio, el JavaScript que inyectes debe llamar a `alert()` para mostrar las cookies de la víctima. En ejercicios posteriores, harás que el ataque haga cosas más maliciosas. Antes de comenzar, debes restaurar la versión original de `zoobar/templates/users.html`.

Para este ejercicio, imponemos algunas restricciones sobre cómo puedes desarrollar tu exploit. En particular:

- Tu ataque no puede involucrar ningún cambio en zoobar.
- Tu ataque no puede depender de la presencia de ninguna cuenta de zoobar que no sea la de la víctima.
- Tu solución debe ser una URL que comience con `http://localhost:8080/zoobar/index.cgi/users?`.

Cuando termines, corta y pega tu URL en la barra de direcciones de un usuario con sesión iniciada, y debería imprimir las cookies de la víctima (no olvides iniciar el servidor de zoobar: `./zookld`). Una vez que funcione, coloca tu URL de ataque en un archivo llamado `answer-3.txt`. Tu URL debe ser lo único en la primera línea del archivo.

Hint: Necesitarás encontrar una vulnerabilidad de cross-site scripting en `/zoobar/index.cgi/users`, y luego usarla para inyectar código JavaScript en el navegador. ¿Qué parámetros de entrada de la solicitud HTTP muestra la página resultante `/zoobar/index.cgi/users?` ¿Cuál de ellos no está correctamente escapado?

Hint: ¿Se refleja este parámetro de entrada literalmente de vuelta en el navegador de la víctima? ¿Qué podrías poner en el parámetro de entrada para que el navegador de la víctima ejecute la entrada reflejada? Recuerda que el servidor HTTP realiza la decodificación de la URL en tu solicitud antes de pasársela a zoobar; asegúrate de que tu código de ataque esté codificado en la URL (por ejemplo usa `+` en lugar de espacio, y `%2b` en lugar de `+`). Esta referencia de codificación de URL y esta herramienta de conversión pueden ser útiles.

Hint: El navegador puede cachear los resultados de cargar tu URL, así que querrás asegurarte de que la URL sea siempre diferente mientras la desarrollas. Puede que quieras poner un argumento aleatorio en tu URL: `&random=<algún número aleatorio>`.

Ejercicio 4: Robar cookies.

Modifica la URL para que no imprima las cookies sino que las registre para el atacante. Coloca tu URL de ataque en un archivo llamado `answer-4.txt`.

Hint: Incorpora tu script de registro del ejercicio 2 en la URL.

Ejercicio 5: Ocultar tus huellas.

Con los exploits que has desarrollado hasta ahora, es probable que la víctima note que robaste sus cookies, o al menos, que algo extraño está sucediendo. Por ejemplo, es probable que la página de Users también haya mostrado un mensaje de error (por ejemplo, “Cannot find that user”).

Para este ejercicio, necesitas modificar tu URL para ocultar tus huellas. Excepto por la barra de direcciones del navegador (que puede ser diferente), el calificador debe ver una página que se vea **exactamente** igual a cuando

visita <http://localhost:8080/zoobar/index.cgi/users>. No deben ser visibles cambios en la apariencia del sitio ni texto adicional. Evitar el *texto de advertencia en rojo* es una parte importante de este ataque (está bien si la página se ve rara brevemente antes de corregirse). Tu script aún debe enviar la cookie del usuario al script de registro.

Cuando termines, coloca tu URL de ataque en un archivo llamado **answer-5.txt**.

Hint: Probablemente quieras usar CSS para hacer que tus ataques sean invisibles para el usuario. Familiarízate con expresiones básicas como `<style>.warning{display:none}</style>`, y siéntete libre de usar atributos sigilosos como `display: none`; `visibility: hidden`; `height: 0`; `width: 0`; y `position: absolute`; en el HTML de tus ataques. Ten cuidado, ya que los frames e imágenes pueden comportarse de forma extraña con `display: none`, por lo que podrías preferir usar `visibility: hidden`.

Nota: no hay ejercicios 6-8 en este laboratorio; el siguiente ejercicio empieza en el número 9.

Parte 2: Página de inicio de sesión falsa

Los ataques en línea más sofisticados a menudo explotan múltiples vectores de ataque. En esta parte, construirás un ataque que (1) robará los zoobars de una víctima si el usuario ya ha iniciado sesión (usando el ataque del ejercicio 8), o (2) robará el nombre de usuario y la contraseña de la víctima si no ha iniciado sesión usando un formulario de inicio de sesión falso. Al igual que en la última parte del laboratorio, el escenario del ataque es que logramos que el usuario visite alguna página web maliciosa que controlamos. En esta parte del laboratorio, primero construiremos el ataque de robo de información de inicio de sesión, y luego combinaremos ambos en una sola página maliciosa.

Ejercicio 9: Crear un formulario de inicio de sesión de zoobar

Copia el formulario de inicio de sesión de zoobar (ya sea viendo el código fuente de la página o usando `zoobar/templates/login.html`) en **answer-9.html**, y haz que funcione con el sitio existente de zoobar. Gran parte de esto implicará anteponer prefijos a las URL con la dirección del servidor web. Este archivo se usará como punto de partida para el resto de los ejercicios en esta parte, así que asegúrate de poder iniciar sesión correctamente en el sitio web usando tu formulario falso. Ten en cuenta que **no** debes hacer cambios en el código de zoobar. Envía tu HTML en un archivo llamado **answer-9.html**.

Ejercicio 10: Interceptar el envío del formulario

Para robar las credenciales de la víctima, tenemos que observar los valores del formulario justo cuando el usuario está enviando el formulario. Esto se hace fácilmente adjuntando un event listener (usando `addEventListener()`) o configurando el atributo `onsubmit` de un formulario. Para este ejercicio, usa uno de estos métodos para mostrar la contraseña del usuario cuando se envíe el formulario. Envía tu código en un archivo llamado **answer-10.html**.

Ejercicio 11: Robar la contraseña

Modifica **answer-10.html** para registrar el nombre de usuario y la contraseña (separados por una barra) usando el script de registro cuando el usuario envíe el formulario de inicio de sesión. Envía tu código en un archivo llamado **answer-11.html**.

Ten en cuenta que después de implementar este ejercicio, la página del controlador del atacante ya no redirigirá al usuario para que inicie sesión correctamente. Corregirás este problema en el ejercicio 12.

Hint: Cuando se envía un formulario, las solicitudes pendientes se cancelan mientras el navegador navega a la nueva página. Esto podría hacer que tu solicitud a `log.php` no se complete. Para evitarlo, considera cancelar el envío del formulario usando el método `preventDefault()` en el objeto de evento pasado al manejador de envío, y luego usa `setTimeout()` para enviar el formulario de nuevo un poco más tarde. ¡Recuerda que tu manejador de envío podría invocarse otra vez!

Ejercicio 12: Oculta tus huellas

Modifica **answer-11.html** para ocultar tus huellas: organiza que, después de robar el nombre de usuario y la contraseña de la víctima, el usuario vea el sitio oficial. Envía tu código en un archivo llamado **answer-12.html**.

Hint: La aplicación de zoobar verifica *cómo* se envió el formulario (es decir, si se hizo clic en “Log in” o en “Register”) observando si los parámetros de la solicitud contienen `submit_login` o `submit_registration`. Ten esto en mente cuando reenvíes el intento de inicio de sesión a la página real.

Ejercicio 13 (opcional): Canales laterales y phishing.

Modifica **answer-12.html** para que tu JavaScript robe los zoobars de la víctima si el usuario ya ha iniciado sesión, o de lo contrario siga el ejercicio 12: pide a la víctima su nombre de usuario y contraseña si no ha iniciado sesión y

roba la contraseña de la víctima. Como en el ejercicio anterior, asegúrate de **no** cargar el archivo **answer-13.html** desde **http://localhost:8080/**.

El script de calificación ejecutará el código una vez mientras la sesión está iniciada en el sitio de zoobar antes de cargar tu página. Luego ejecutará el código una segunda vez mientras *no* haya sesión iniciada en el sitio de zoobar antes de cargar tu página. En consecuencia, cuando el navegador cargue tu documento, tu documento malicioso debe detectar si el usuario ha iniciado sesión en el sitio de zoobar. Envía tu documento HTML final en un archivo llamado **answer-13.html**.

Hint: La política del mismo origen generalmente no permite que tu página de ataque acceda al contenido de páginas de otro dominio. ¿Qué tipos de archivos puede cargar tu página de ataque desde otro dominio? ¿Tiene la aplicación web de zoobar archivos de ese tipo? ¿Cómo puedes inferir si el usuario ha iniciado sesión o no, basándote en esto?

Parte 3: Gusano de perfiles

Los gusanos en el contexto de la seguridad web son scripts que son inyectados en páginas por un atacante y que se propagan automáticamente una vez que se ejecutan en el navegador de una víctima. El gusano Samy es un excelente ejemplo, que se propagó a más de un millón de usuarios en la red social MySpace en tan solo 20 horas. En esta parte del laboratorio, crearás un gusano similar que, al ejecutarse, transferirá 1 zoobar de la víctima al atacante y luego se propagará al perfil de la víctima. Así, cualquier visita posterior al perfil de la víctima hará que se transfieran zoobars adicionales y que el gusano se propague nuevamente. Construirás tu solución en varias etapas, como en las partes anteriores. Esta vez, sin embargo, no detallaremos los pasos mediante ejercicios.

Ejercicio 14: Gusano de perfiles.

Tu gusano de perfil debe enviarse en un archivo llamado **answer-14.txt**. Para calificar tu ataque, copiaremos y pegaremos el código del perfil enviado en el perfil del usuario “attacker” y veremos ese perfil usando la cuenta del calificador. Luego veremos el perfil del calificador con más cuentas, verificando tanto la transferencia de zoobars como la replicación del código del perfil.

En particular, requerimos que tu gusano cumpla con los siguientes criterios:

- Cuando se vea el perfil de un usuario infectado, se debe transferir 1 zoobar desde la cuenta del usuario que lo está viendo a la cuenta del usuario llamado “attacker”.
- Cuando un usuario visite un perfil infectado, el gusano (es decir, el código del perfil infectado) debe propagarse al perfil del usuario que lo visita.
- Un perfil infectado debe mostrar el mensaje **Scanning for viruses...** cuando se vea, como si ese fuera todo el contenido del perfil visualizado.
- La transferencia y la replicación deben ser razonablemente rápidas (menos de 15 segundos). Durante ese tiempo, el calificador no hará clic en ninguna parte.
- Durante el proceso de transferencia y replicación, la barra de direcciones del navegador debe permanecer en **http://localhost:8080/zoobar/index.cgi/users?user=**, donde es el usuario cuyo perfil se está viendo. El visitante no debe ver elementos adicionales de interfaz gráfica (por ejemplo, frames), y el usuario cuyo perfil se está viendo debe parecer tener 10 zoobars y ningún registro de transferencias. Estos requisitos hacen que el ataque sea más difícil de detectar para un usuario y, por tanto, más realista, pero también hacen que el ataque sea más difícil de lograr.

Para ayudarte a comenzar, aquí tienes un esquema aproximado de cómo construir tu gusano:

- Crea un perfil para el atacante para familiarizarte con cómo funcionan los perfiles en zoobar. Debes inspeccionar el código fuente para comprender la disposición del HTML, tanto al editar tu propio perfil como al ver el de otra persona.
- Modifica tu perfil para transferir un zoobar del usuario que visita el perfil a la cuenta “attacker”.
- Oculta cualquier rastro que la víctima pueda observar.
- Organiza la replicación del perfil.

Este análisis detallado del gusano de MySpace puede servirte de inspiración.

Note: No se te calificará por el caso extremo en el que el usuario que ve el perfil no tenga zoobars para enviar.

Hint: En este ejercicio, a diferencia de los anteriores, tu exploit se ejecuta en el mismo dominio que el sitio objetivo. Esto significa que no estás sujeto a las restricciones de la política de mismo origen, y que puedes emitir solicitudes AJAX directamente usando XMLHttpRequest en lugar de iframes.

Hint: En este ejercicio, puede que necesites crear nuevos elementos en la página y acceder a datos dentro de ellos. Los métodos del DOM `document.createElement` y `document.body.appendChild` pueden resultarte útiles para este propósito.

Hint: Si eliges usar `iframes` en tu solución, quizá quieras acceder a campos de formularios dentro de un `iframe`. La manera exacta de hacerlo varía según el navegador, pero dicho acceso siempre está restringido por la política de mismo origen. En Firefox, puedes usar `iframe.contentDocument.forms[0].some_field_name.value = 1;`.

Entregables

Asegúrate de tener los siguientes archivos:

`answer-1.js`, `answer-2.js`, `answer-3.txt`, `answer-5.txt`, `answer-6.html`, `answer-7.html`, `answer-8.html`, `answer-9.html`, `answer-10.html`, `answer-11.html`, `answer-12.html`, `answer-13.html`, `answer-14.txt`, que contengan cada uno de tus ataques. Siéntete libre de incluir cualquier comentario sobre tus soluciones en el archivo `answers.txt` (agradeceremos cualquier retroalimentación que puedas tener sobre esta tarea).

Envía tus respuestas a la tarea del laboratorio ejecutando `make prepare-submit` y sube el archivo resultante `lab3-handin.tar.gz` a Canvas.

Agradecimientos

CS155